

```
import pandas as pd
import numpy as np

df = pd.read_csv('/content/Dataset.csv')

print(df.shape)
print(df.columns.tolist())
df.head()
```

(3900, 18)
['Customer ID', 'Age', 'Gender', 'Item Purchased', 'Category', 'Purchase Amount (USD)', 'Location', 'Size', 'Color', 'Season', 'Review Rating', 'Subscription Status', 'Shipping Type', 'Discount Applied', 'Promo Code Used', 'Previous Purchases', 'Payment Method', 'Frequency of Purchases']

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping

```
df.isnull().sum()
```

	0
Customer ID	0
Age	0
Gender	0
Item Purchased	0
Category	0
Purchase Amount (USD)	0
Location	0
Size	0
Color	0
Season	0
Review Rating	37
Subscription Status	0
Shipping Type	0
Discount Applied	0
Promo Code Used	0
Previous Purchases	0
Payment Method	0
Frequency of Purchases	0

dtype: int64

```
df['Review Rating'].fillna(df['Review Rating'].median(), inplace=True)
df.isnull().sum() # should show all zeros now
```

/tmp/ipykernel_3519/4164887442.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through cha
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are se
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] =

```
df['Review Rating'].fillna(df['Review Rating'].median(), inplace=True)
```

	0
Customer ID	0
Age	0
Gender	0
Item Purchased	0
Category	0
Purchase Amount (USD)	0
Location	0
Size	0
Color	0
Season	0
Review Rating	0
Subscription Status	0
Shipping Type	0
Discount Applied	0
Promo Code Used	0
Previous Purchases	0
Payment Method	0
Frequency of Purchases	0

dtype: int64

```
print("Before:", df.shape)
df.drop_duplicates(inplace=True)
print("After:", df.shape)
```

Before: (3900, 18)
After: (3900, 18)

```
# Check current types first
df.dtypes
```

0	
Customer ID	int64
Age	int64
Gender	object
Item Purchased	object
Category	object
Purchase Amount (USD)	int64
Location	object
Size	object
Color	object
Season	object
Review Rating	float64
Subscription Status	object
Shipping Type	object
Discount Applied	object
Promo Code Used	object
Previous Purchases	int64
Payment Method	object
Frequency of Purchases	object

dtype: object

```
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
df['Purchase Amount (USD)'] = pd.to_numeric(df['Purchase Amount (USD)'], errors='coerce')
df['Review Rating'] = pd.to_numeric(df['Review Rating'], errors='coerce')
df['Previous Purchases'] = pd.to_numeric(df['Previous Purchases'], errors='coerce')
```

```
text_cols = ['Gender', 'Item Purchased', 'Category', 'Location',
             'Size', 'Color', 'Season', 'Subscription Status',
             'Shipping Type', 'Discount Applied',
             'Promo Code Used', 'Payment Method', 'Frequency of Purchases']

for col in text_cols:
    df[col] = df[col].str.strip().str.title()
```

```
print(df['Age'].describe())
print(df['Purchase Amount (USD)'].describe())
print(df['Review Rating'].describe())
```

```
count    3900.000000
mean      44.068462
std       15.207589
min       18.000000
25%       31.000000
50%       44.000000
75%       57.000000
max       70.000000
Name: Age, dtype: float64
count    3900.000000
mean      59.764359
std       23.685392
min       20.000000
25%       39.000000
50%       60.000000
75%       81.000000
max      100.000000
Name: Purchase Amount (USD), dtype: float64
count    3900.000000
mean       3.750538
std        0.713589
min        2.500000
25%        3.100000
50%        3.800000
75%        4.400000
```

```
max      5.000000
Name: Review Rating, dtype: float64
```

```
df.to_csv('cleaned_data.csv', index=False)
print("Done! Shape:", df.shape)
```

```
Done! Shape: (3900, 18)
```

```
#1. Promo Dependency Score
df['promo_dependency_score']=((df['Promo Code Used'].str.lower()=='yes').astype(int)+(df['Discount Applied'].str.lower()=='ye
```

```
#Value Tier
df['value_tier']=pd.qcut(df['Purchase Amount (USD)'],q=3,labels=['Low','Mid','High'])
```

```
#3. Satisfaction Flag
df['Satisfaction_flag']=(df['Review Rating']>=4).astype(int)
```

```
#4. Loyalty Definition 1 – Frequency + Promo Independence
freq_map={'Weekly':4,'Fortnight':2,'Bi-weekly':2,'Monthly':1,'Quarterly':0.33,'Annually':0.08,'Every 3 Months': 0.33}
df['freq_score']=df['Frequency of Purchases'].str.title().map(freq_map)

df['loyal_def1']=((df['Previous Purchases']>=df['Previous Purchases'].median())&(df['promo_dependency_score']<0.5)).astype(in
```

```
#5.Loyalty Definition 2-Spend + Satisfaction
df['loyal_def2']=((df['value_tier']=='High')&(df['Satisfaction_flag']==1)).astype(int)
```

```
#6.Avg Spend Proxy (spend per purchase history unit)
df['spend per purchase history']=df['Purchase Amount (USD)']/(df['Previous Purchases']+1)
```

```
#7.Subscriber Premium Flag
df['is subscriber']=(df['Subscription Status'].str.lower()=='yes').astype(int)
```

```
print(df[['value_tier', 'promo_dependency_score', 'loyal_def1', 'loyal_def2']].value_counts().head(10))
print("\nLoyal Def1 count:", df['loyal_def1'].sum())
print("Loyal Def2 count:", df['loyal_def2'].sum())
```

value_tier	promo_dependency_score	loyal_def1	loyal_def2	
Low	1.0	0	0	566
Mid	1.0	0	0	563
	0.0	1	0	386
Low	0.0	0	0	371
		1	0	363
Mid	0.0	0	0	354
High	1.0	0	0	314
			1	234
	0.0	1	0	215
		0	0	206

```
Name: count, dtype: int64
```

```
Loyal Def1 count: 1142
Loyal Def2 count: 562
```

```
df.to_csv('feature_engineered.csv', index=False)
print("Feature engineering done! Shape:", df.shape)
```

```
Feature engineering done! Shape: (3900, 26)
```

Loyal Def1: 1,142 customers — high purchase history + no promo dependency. These are our organically retained customers. Loyal Def2: 562 customers — high spenders who are also satisfied. Smaller, more premium segment. The two definitions give different cuts of loyalty, which is exactly what the problem wants. We will argue for one later.

SQL QUERIES

```
import sqlite3
import pandas as pd
df=pd.read_csv('feature_engineered.csv')
conn=sqlite3.connect(':memory:')
```

```
df.to_sql('customers',conn,index=False,if_exists='replace')
print('database ready')
```

database ready

```
q1 = """
SELECT
    customer_segment,
    COUNT(*) AS customer_count,
    ROUND(AVG("Previous Purchases"),2) AS avg_prev_purchases,
    ROUND(AVG("Purchase Amount (USD)"),2) AS avg_spend,
    ROUND(AVG(promo_dependency_score),2) AS avg_promo_dependency,
    ROUND(AVG("Review Rating"),2) AS avg_rating
FROM
(
    SELECT *,
        CASE
            WHEN "Purchase Amount (USD)" < 40 THEN 'Low'
            WHEN "Purchase Amount (USD)" <= 75 THEN 'Mid'
            ELSE 'High'
        END AS customer_segment
    FROM customers
) t
GROUP BY customer_segment
ORDER BY avg_spend DESC
"""
pd.read_sql_query(q1, conn)
```

	customer_segment	customer_count	avg_prev_purchases	avg_spend	avg_promo_dependency	avg_rating
0	High	1214	25.36	88.11	0.42	3.78
1	Mid	1672	25.44	57.53	0.43	3.73
2	Low	1014	25.19	29.52	0.44	3.76

```
q2 = """
SELECT
    Season,
    Category,
    ROUND(AVG("Previous Purchases"), 2) as avg_prev_purchases,
    COUNT(*) as customer_count
FROM customers
GROUP BY Season, Category
ORDER BY avg_prev_purchases DESC
"""
pd.read_sql_query(q2, conn)
```

	Season	Category	avg_prev_purchases	customer_count
0	Summer	Accessories	26.80	312
1	Winter	Footwear	26.66	140
2	Winter	Clothing	26.07	448
3	Spring	Outerwear	25.95	81
4	Spring	Accessories	25.94	301
5	Summer	Footwear	25.61	160
6	Winter	Accessories	25.55	303
7	Fall	Clothing	25.45	427
8	Winter	Outerwear	25.18	80
9	Spring	Clothing	24.76	454
10	Fall	Accessories	24.65	324
11	Fall	Footwear	24.59	136
12	Summer	Clothing	24.46	408
13	Fall	Outerwear	24.39	88
14	Summer	Outerwear	24.32	75
15	Spring	Footwear	24.17	163

```

q3="""
SELECT
    Location,
    COUNT(*) as customer_count,
    ROUND(AVG("Purchase Amount (USD)"),2) as avg_purchase_amt,
    ROUND(AVG(promo_dependency_score),2) as avg_promo_dependency,
    ROUND(
        SUM(
            CASE
                WHEN "Discount Applied"='Yes'
                THEN 1
                ELSE 0
            END
        )*100.0/COUNT(*),1
    ) as pct_discount_applied
FROM customers
GROUP BY Location
ORDER BY avg_promo_dependency ASC
LIMIT 15
"""
pd.read_sql_query(q3, conn)

```

	Location	customer_count	avg_purchase_amt	avg_promo_dependency	pct_discount_applied
0	Kansas	63	54.56	0.24	23.8

```
q4="""
SELECT
CASE
    WHEN loyal_def1=1 THEN 'organically loyal'
    WHEN promo_dependency_score = 1.0 THEN 'Discount Dependent'
    ELSE 'neutral'
END AS customer_type,
COUNT(*) as count,
ROUND(AVG("Purchase Amount (USD)"), 2) as avg_spend,
ROUND(AVG("Previous Purchases"), 2) as avg_purchases,
ROUND(AVG("Review Rating"), 2) as avg_rating
FROM customers
GROUP BY customer_type
ORDER BY avg_spend DESC
"""
pd.read_sql_query(q4, conn)
```

11	Alaska	customer_type	count	72	avg_spend	67.60	avg_purchases	avg_rating	0.40	40.3
0	organically loyal	1142	60.80	37.22	3.79	0.40	40.2			
13	Michigan	neutral	1081	73	59.43	62.10	12.21	3.73	0.40	39.7
2	Discount Dependent	1677	59.28	25.74	3.74	0.40	40.3			

```
q5 = """
SELECT
    Gender,
    ROUND(AVG(Age), 0) as avg_age,
    Category,
    "Payment Method",
    "Subscription Status",
    COUNT(*) as count,
    ROUND(AVG("Purchase Amount (USD)"), 2) as avg_spend,
    ROUND(AVG("Previous Purchases"), 2) as avg_purchases,
    ROUND(AVG(promo_dependency_score), 2) as promo_dependency
FROM customers
WHERE value_tier = 'High' AND loyal_def2 = 1
GROUP BY Gender, Category, "Payment Method", "Subscription Status"
ORDER BY avg_spend DESC
"""
pd.read_sql_query(q5, conn)
```

	Gender	avg_age	Category	Payment Method	Subscription Status	count	avg_spend	avg_purchases	promo_dependency
0	Male	27.0	Outerwear	Cash	No	2	99.50	23.00	0.00
1	Male	53.0	Outerwear	Paypal	No	6	93.67	26.50	0.17
2	Female	58.0	Footwear	Credit Card	No	4	93.25	18.00	0.00
3	Female	39.0	Outerwear	Debit Card	No	4	92.75	30.00	0.00
4	Male	37.0	Footwear	Cash	Yes	8	92.13	27.25	1.00
...	...	...	...	...	...	...	...	...	...